

Átomos de confusão em SystemVerilog

Ideias, regras da linguagem e hipóteses dos seis casos

Este guia explica os pares de código da versão **lambda** (com átomo) e da versão **omega** (sem átomo). “Átomo” designa aqui uma construção candidata a dificultar a compreensão. A dificuldade é uma hipótese a investigar, não uma conclusão antecipada sobre os participantes.

O critério do experimento

Os dois lados devem produzir o mesmo output **nas entradas e condições escolhidas para cada questão**. Não se exige equivalência para todas as entradas. Exemplos alternativos ajudam a discutir consequências no artigo, sem invalidar esse critério.

Caso	Foco técnico	Output nas duas versões
1	Base decimal implícita em rótulos de case	out = 30
2	Concatenação de bits versus atribuição por elemento	out0 = 3; out1 = 0
3	Tempo de vida implícito da função	r1 = r2 = r3 = 7*
4	Soma com operandos signed e unsigned	out = 6
5	Comparações relacionais encadeadas	selected = 164
6	Curingas simétricos e assimétricos	opcode = 001

* Caso 3: resultado confirmado pelo responsável no seu simulador. A distinção entre essa observação e as regras de inicialização é tratada nas páginas 4 e 5.

Como ler

Cada caso separa **construção observada**, **regra técnica**, **percurso do exemplo** e **hipótese cognitiva**. Os trechos mostram apenas as linhas relevantes; as declarações e condições omitidas são informadas no texto.

Algumas denominações iniciais precisam de precisão: os rótulos atuais do caso 1 são todos decimais; o caso 2 não contém duas concatenações; e, no caso 3, static trata de tempo de vida, não do tipo de retorno. As páginas seguintes preservam a intenção do estudo e ajustam esses termos.

Roteiro: casos 1 e 2 (p. 2-3); caso 3 (p. 4-5); casos 4 e 5 (p. 6-7); caso 6 (p. 8-9); interpretação experimental (p. 10); referências (p. 11).

O que parece binário pode ser decimal

Definição operacional do átomo

Neste estudo, o caso permanece sempre dentro de um **case**: um seletor binário é comparado com rótulos cuja escrita pode induzir uma leitura de base diferente da real. A regra de base numérica também existe fora de case; a restrição ao case é do recorte experimental.

```
// Com átomo
case (selector)
  0: decoded = 8'd10;
  01: decoded = 8'd20;
  2: decoded = 8'd30;
  10: decoded = 8'd40;
  default: decoded = 8'd100;
endcase

// Sem átomo
case (selector)
  2'b00: decoded = 8'd10;
  2'b01: decoded = 8'd20;
  2'b10: decoded = 8'd30;
  2'b11: decoded = 8'd40;
  default: decoded = 8'd100;
endcase
```

A precisão que importa

No código com átomo, **0, 01, 2 e 10 são todos literais decimais**. O zero à esquerda de 01 não muda sua base. Assim, não há, nos rótulos desse print, mistura literal de decimal e binário. Há uma expectativa de leitura binária que pode entrar em conflito com a base decimal implícita. [1]

Uma mistura real de bases seria, por exemplo, usar 0, 2'b01, 2 e 2'b11 no mesmo case. Isso é permitido: case compara valores, não a aparência do texto. Para o artigo, o nome mais fiel ao exemplo atual é **“base implícita e aparência binária nos rótulos de case”**.

Como o exemplo chega a 30

O seletor tem dois bits. A entrada 2'b10 representa o número decimal 2. Na versão lambda, ela encontra o rótulo 2; em omega, encontra 2'b10. Ambos atribuem 30 a decoded e, com enable = 1, transferem 30 para out.

Entrada, enable = 1	Com átomo	Sem átomo
2'b00	10	10
2'b01	20	20
2'b10 (questão)	30	30
2'b11 (outro cenário)	100	40

O rótulo decimal 10 exige o valor dez, que não cabe no seletor de dois bits. O seletor é alargado para a comparação; o literal não é simplesmente truncado para dois bits. Esse ramo é inalcançável neste módulo. [1]

Hipótese de confusão

O participante pode ler o rótulo 10 como binário e escolher out = 40. A escrita 2'b10 explicita largura e base. O estudo pode investigar se essa explicitação reduz tempo e tentativas; não precisa afirmar que usar bases diferentes seja, por si só, um erro.

Bits reunidos não são elementos inicializados

```
logic [1:0][31:0] A;

// Com átomo                // Sem átomo
A = {1'b1, 1'b1};           A[0] = 2'b11;
                              A[1] = 2'b00;

out0 = A[0];
out1 = A[1];
```

O átomo proposto

A construção compacta **{1'b1, 1'b1}** pode parecer uma lista que atribui um valor a cada posição de A. Na realidade, ela concatena dois bits. A versão sem átomo escreve diretamente o valor de cada elemento. Portanto, são duas formas de obter o mesmo conteúdo de A, mas **somente a primeira é concatenação**.

Passo a passo no array packed

As duas dimensões aparecem antes do nome A: é um array *packed* de 64 bits. A[1] corresponde aos 32 bits mais significativos; A[0], aos 32 menos significativos. A concatenação tem a soma das larguras de seus termos: 1 + 1 = 2 bits. Seu valor é 2'b11, isto é, 3. Ao atribuí-la a A, os bits altos são preenchidos com zeros. [2, 3]

```
{1'b1, 1'b1} = 2'b11

A[1] (32 bits altos)      A[0] (32 bits baixos)
00000000000000000000000000000000 | 00000000000000000000000000000011

out1 = 0                  out0 = 3
```

O apóstrofo muda a operação

Expressão	Interpretação para este A	Resultado
{1'b1, 1'b1}	Concatena dois bits e alarga o conjunto	A[1] = 0; A[0] = 3
'{1'b1, 1'b1}	Padrão de atribuição: ajusta cada item ao elemento correspondente	A[1] = 1; A[0] = 1

O padrão de atribuição **'{...}** é sensível à estrutura de destino; ele não é um nome alternativo para concatenação. Aqui, cada item é ajustado a um elemento de 32 bits. A diferença visual é pequena, mas o resultado muda. [2]

Relação com outras linguagens e hipótese

A semelhança mais precisa é com **listas de inicialização de agregados**, como as de C/C++, e com a própria sintaxe de padrão de atribuição do SystemVerilog. Não se deve generalizar que chaves tenham esse significado em qualquer linguagem: em Python, por exemplo, chaves podem construir conjuntos ou dicionários.

A hipótese é que alguém interprete os dois itens como “um para cada elemento”, prevendo out0 = 1 e out1 = 1. A atribuição por índice evidencia a distribuição real dos bits e reduz essa inferência implícita. O efeito precisa ser medido; a concisão da concatenação não prova dificuldade por si só.

A função guarda estado entre as chamadas?

Definição operacional do átomo

Omissão do qualificador de tempo de vida da função, deixando o leitor recuperar a regra do contexto. A versão explícita acrescenta `static`. O tipo de retorno continua sendo `int` nas duas versões.

```
// Com átomo
function int maxx(int a, int b);

// Sem átomo (o corpo é o mesmo)
function static int maxx(int a, int b);
    int max = a;
    if (b > max)
        max = b;
    return max;
endfunction

r1 = maxx(3, 7);
r2 = maxx(1, 2);
r3 = maxx(0, 0);
```

Static e automatic

Tempo de vida descreve por quanto tempo existe o armazenamento de argumentos e variáveis locais. Com tempo de vida estático, há armazenamento compartilhado entre chamadas. Com tempo de vida automático, cada ativação recebe seu armazenamento local, adequado também a chamadas simultâneas ou recursivas. Declarações locais explícitas podem ter regras próprias. [4]

No módulo comum do exemplo, omitir o qualificador mantém o padrão estático. Acrescentar `static` não deveria transformar essa função em outro algoritmo: torna visível uma propriedade que já era implícita. Trocar por `automatic` seria uma mudança de comportamento, não a versão sem átomo escolhida para este estudo.

As duas hipóteses de leitura

Leitura do participante	Resultado previsto
Supõe uma nova inicialização de <code>max</code> a cada chamada, como numa função automática	7, 2, 0
Reconhece o estado persistente e considera <code>max</code> já igual a 7 após a primeira chamada	7, 7, 7

Depois que `max` contém 7, as comparações seguintes, $2 > 7$ e $0 > 7$, são falsas. O valor permanece 7. O que persiste é a **variável `max`**; o código não lê `r1` para calcular `r2` nem lê `r2` para calcular `r3`.

Resultado observado e limite da explicação

O responsável confirmou 7, 7, 7 nas duas versões no simulador usado. Essa observação sustenta o gabarito adotado. Ela não autoriza dizer que “a variável estática é inicializada na primeira chamada”: a inicialização estática tem regras próprias, detalhadas na próxima página.

O contexto muda o padrão; a posição de static importa

Contexto	Tempo de vida da função / método
Função em módulo comum, sem qualificador	Estático por padrão
Função em módulo declarado automatic, sem sobrescrita local	Automático, herdado do contexto
Função com function automatic em módulo	Automático explícito
Método definido em classe	Automático; tempo de vida estático do método é proibido
static function dentro de classe	Método associado à classe; tempo de vida continua automático

Essas distinções permitem uma hipótese plausível: alguém acostumado a métodos de classe pode transportar a expectativa de variáveis locais automáticas para uma função de módulo, sem perceber a mudança de contexto. Mesmo uma classe declarada dentro de um módulo continua tendo as regras de métodos de classe. [4]

Duas posições, dois significados

```
// Em módulo: qualificador de tempo de vida
function static int f();

// Em classe: qualificador de membro da classe
static function int f();
```

No segundo caso, static permite um método ligado à classe, sem depender de uma instância específica. Isso não torna seus argumentos e variáveis locais automaticamente persistentes. Tampouco se deve confundir variáveis locais automáticas com propriedades do objeto, que podem preservar estado entre chamadas. [4]

A inicialização não é uma atribuição comum

Na declaração **int max = a;**, o “= a” é um inicializador. Para uma variável de tempo de vida estático, a inicialização ocorre antes dos processos initial/always, não toda vez que a função é chamada. Já separar **int max;** de **max = a;** faz a atribuição executar em cada chamada. [5]

Por isso, a primeira chamada merece atenção: quando a inicialização estática lê a, os argumentos dessa chamada ainda não foram passados. Em simulação de quatro estados, X pode chegar a max, e b > max pode não produzir uma condição verdadeira. Além disso, inicialização de variável implicitamente estática pode gerar diagnóstico de conformidade. Não é correto deduzir universalmente 7, 7, 7 apenas da palavra static. [5]

Como apresentar no artigo

Separar duas afirmações: (1) a hipótese é que explicitar static ajude a reconhecer persistência; (2) no ambiente de simulação utilizado, o exemplo produziu 7, 7, 7. Documentar simulador, versão e opções torna a segunda afirmação reproduzível, sem substituir a regra geral de inicialização.

Não foi realizada uma nova simulação para este guia. O resultado experimental do caso 3 é o informado pelo responsável. Nenhuma alteração do exemplo é proposta aqui.

Somar valores com sinais de tipos diferentes

```
byte in = 5;           // 8 bits, signed
int result;           // 32 bits, signed

// Com átomo           // Sem átomo
result = in + 1'b1;     result = in + 1;
```

O átomo deste estudo é a **soma** de um operando signed com outro unsigned, seguida da interpretação das conversões implícitas. A operação permanece sempre uma adição no recorte experimental, embora as regras de tipos tenham efeitos também em outros operadores.

O que muda entre os literais

O literal 1 é decimal sem base explícita e signed. O literal 1'b1 tem um bit e é unsigned. Na expressão mista do exemplo, a avaliação é unsigned; o contexto de result, de 32 bits, influencia a largura, mas seu tipo signed não “conserta” o sinal da soma à direita. [1, 6]

Com in = 5, a extensão por zeros ou por sinal dá o mesmo valor positivo. Assim, ambas as somas produzem 6 e, para enable = 1, out recebe 6. O exemplo esconde a consequência numérica da diferença de tipos, permitindo estudar sua leitura.

Por que -5 evidencia o problema

Etapa	in + 1'b1	in + 1
Bits de in, em 8 bits	11111011	11111011
Ampliação para o contexto de 32 bits	Zeros à esquerda: valor 251	Sinal repetido: valor -5
Resultado da soma	251 + 1 = 252	-5 + 1 = -4

Não é um erro do operador +: o cálculo segue os tipos e larguras da expressão. A surpresa é interpretar o padrão 11111011 como -5, mas permitir que a ampliação unsigned o transforme no valor positivo 251 antes da soma.

Padronizar signed + signed ou unsigned + unsigned?

Sim, conforme o domínio. Para valores não negativos, operandos unsigned com larguras adequadas podem ser coerentes. Para preservar o significado matemático de -5 nesta soma e obter -4 em int, a solução direta é signed + signed, como in + 1 ou in + 32'sd1. Não basta uniformizar o sinal sem também considerar largura e overflow.

Um unsigned de oito bits pode guardar os mesmos bits de -5, mas seu valor numérico será 251. É possível realizar aritmética modular unsigned e reinterpretar bits depois; isso é outro contrato de representação, não uma conversão que preserve automaticamente o número negativo.

Armadilha adicional

Não substituir 1'b1 por 1'sb1 esperando “+1 signed”: em um bit com sinal, o padrão 1 representa **-1**. Para +1 signed, use largura suficiente, como 2'sb01 ou 32'sd1.

Hipótese: o leitor pode tratar todos os literais com valor aparente 1 como intercambiáveis, ignorando sinal e largura. A versão padronizada reduz essa exigência de inferência.

Uma cadeia visual não é um teste de intervalo

```
lo = 20;
med = 164;
hi = 224;

// Com átomo
result = (lo < med < hi);

// Sem átomo
result = (lo < med) && (med < hi);
```

A diferença semântica

Em SystemVerilog, os operadores relacionais encadeados desse trecho associam-se à esquerda: a expressão é agrupada como **(lo < med) < hi**. Primeiro surge o resultado de uma comparação; depois, esse resultado é comparado com hi. Não é uma abreviação de duas desigualdades ligadas por “e”. [3]

Etapa com os valores da questão	Com átomo	Sem átomo
Primeira comparação	$20 < 164 = 1$	$20 < 164 = 1$
Segunda comparação	$1 < 224 = 1$	$164 < 224 = 1$
Resultado final	1	$1 \ \&\& \ 1 = 1$
Ramo executado	selected = med = 164	selected = med = 164

A expectativa trazida de Python

Em Python, $lo < med < hi$ expressa a cadeia de comparações esperada na matemática: ambas precisam ser verdadeiras, e o termo intermediário é avaliado uma única vez. SystemVerilog não atribui esse significado especial à mesma aparência textual. [7]

É semelhante ao caso 2 como hipótese de transferência de conhecimento: uma forma familiar convida a uma interpretação que não corresponde à operação real. Aqui a analogia com Python é específica e verificável; não é apenas uma semelhança genérica entre símbolos.

Outro cenário, fora da questão

Com $lo = 20$, $med = 224$ e $hi = 164$, o encadeamento calcula $(20 < 224) < 164$, isto é, $1 < 164$, e seleciona 224. A versão com $\&\&$ exige também $224 < 164$; como isso é falso, seleciona 164. Os valores crescentes do experimento fazem os outputs coincidirem, mas não tornam as expressões equivalentes em geral.

Experiência não elimina a hipótese

Um usuário experiente pode escrever o encadeamento sabendo que compara um resultado booleano com hi; um iniciante pode interpretá-lo como intervalo. Também é possível um especialista transferir automaticamente a leitura matemática. A experiência é uma variável a analisar, não garantia de qual modelo mental será usado.

Consequência para a medição

Neste exemplo crescente, uma leitura correta e uma leitura “como Python” podem levar à mesma alternativa. Acertar selected = 164 não demonstra, sozinho, que o participante entendeu a avaliação encadeada. Tempo, tentativas e fixações também precisam ser interpretados com esse limite.

A forma explícita usa $\&\&$ (conjunção lógica), não $\&$ (operação bit a bit). Os parênteses isolam as duas condições pretendidas. Com operandos desconhecidos, há ainda regras de quatro estados; os valores desta questão são conhecidos.

Ignorar um bit do padrão não é ignorar X na entrada

Definição operacional do átomo

Uso de `casex`, que pode transformar desconhecidos da própria entrada em curingas. A versão comparativa escreve decisões `if/else` com `==?`, cuja máscara vem apenas do operando direito. A hipótese envolve perceber de onde vem a permissão para ignorar bits.

Símbolo	Significado necessário para o exemplo
0 e 1	Valores lógicos conhecidos
X	Valor lógico desconhecido na simulação
Z	Alta impedância; também participa das regras de comparação
? em literal binário	Outra escrita de Z; funciona como máscara nos contextos aqui usados, não em qualquer expressão

Três mecanismos que não devem ser confundidos

case comum: compara inclusive X/Z como valores, sem a máscara de `casex`. **casex:** ignora posições com X/Z em qualquer lado da comparação. **==?:** ignora posições com X/Z apenas no padrão à direita. Se um X à esquerda estiver numa posição exigida pelo padrão, a comparação pode produzir X em vez de 0 ou 1. [3, 8, 9]

```
// Com átomo
casex (instruction)
  4'b00?? : opcode = 3'b001;
  4'b0??? : opcode = 3'b010;
  default: opcode = 3'b111;
endcase

// Sem átomo
if (instruction ==? 4'b00??)
  opcode = 3'b001;
else if (instruction ==? 4'b0???)
  opcode = 3'b001;
else
  opcode = 3'b111;
```

Ordem e sobreposição

O padrão `00??` é mais restrito que `0????`: todo valor conhecido que corresponde ao primeiro também corresponde ao segundo. O `casex` executa o primeiro item que combina; a cadeia `if/else` também dá prioridade à primeira condição verdadeira. Inverter a ordem pode alterar o comportamento.

X numa condição não vira automaticamente verdadeiro

Para essas condições de um bit, `if` só entra no ramo quando o resultado é 1; 0, X e Z não o ativam. Logo, uma comparação `==?` que produza X deixa a avaliação seguir para o `else if`. Isso não significa que `==?` retornou 0: o resultado X e a escolha feita por `if` são etapas diferentes. [9]

`==?` também não é `===`. O operador `===` compara X/Z literalmente e retorna 0 ou 1. Em `==?`, a ordem dos operandos importa, porque só o lado direito fornece os curingas.

Mesmo output, percursos diferentes

Entrada da questão: instruction = 4'b0x01

Teste	casex	==? dentro de if
Primeiro padrão: 00??	O X da entrada é ignorado; há correspondência.	O segundo bit exige 0, mas a entrada tem X: resultado X.
Decisão após o primeiro teste	Executa o primeiro ramo: opcode = 001.	if não entra; segue para o segundo teste.
Segundo padrão: 0???	Não é executado, pois o primeiro ramo venceu.	O primeiro bit é 0; os demais são mascarados à direita: resultado 1.
Output final	001 (primeiro ramo)	001 (segundo ramo)

Essa é a razão exata da coincidência. Ela **não demonstra que casex e ==? trataram o X da mesma forma**. Na versão omega, os dois ramos atribuem 001; na versão lambda, o segundo atribui 010. Esses valores fazem parte do exemplo e precisam aparecer na explicação do artigo.

O que acontece com outras entradas

instruction	Com átomo	Sem átomo	Origem da diferença ou igualdade
4'b0001	001	001	Primeiro padrão reconhecido nos dois lados.
4'b0100	010	001	Segundo ramo nos dois lados, mas valores atribuídos diferentes.
4'b0x01	001	001	Primeiro ramo em lambda; segundo em omega.
4'bx001	001	111	casex mascara X; os dois testes ==? dão X e não ativam if.
4'b1001	111	111	Bit mais significativo 1 exclui os dois padrões.

Separar duas causas no artigo

A divergência com 0100 ocorre até sem desconhecidos: ela decorre dos **valores diferentes atribuídos no segundo ramo**. A divergência com x001 evidencia o **tratamento diferente de X**. Usar o primeiro exemplo como prova exclusiva do perigo de casex confundiria duas causas distintas.

O que a versão explícita melhora - e o que não promete

A máscara escrita à direita delimita quais bits podem ser ignorados, reduzindo a leitura simétrica de casex. Ainda assim, if pode encaminhar uma condição X para o else e produzir uma saída conhecida. Portanto, a versão não é um detector universal de desconhecidos; verificações específicas, como \$isunknown, têm outra finalidade.

Hipótese e limite do acerto

Pode haver mais esforço para identificar o ramo de casex ou para interpretar X no if. Porém, como dois percursos distintos chegam a 001, a alternativa correta não identifica o percurso mental. Uma justificativa ou uma pergunta posterior sobre o ramo escolhido ajudaria a distinguir as interpretações.

Como transformar as ideias em afirmações testáveis

Hipóteses por caso

Caso	Mecanismo cognitivo candidato	O que a versão explícita torna visível
1	Ler 10 como binário em vez de decimal	Base e largura dos rótulos
2	Distribuir os itens das chaves por elementos	Qual índice recebe cada valor
3	Esperar armazenamento local novo a cada chamada	Tempo de vida static
4	Tratar 1 e 1'b1 como intercambiáveis	Uma soma com operandos signed
5	Ler o encadeamento como intervalo matemático	Duas comparações ligadas por &&
6	Ignorar a origem dos curingas ou o resultado X	Máscara à direita e sequência de decisões

Igualdade local, não refatoração universal

Uma formulação adequada ao artigo seria: *“Os pares foram construídos para produzir o mesmo resultado nas condições apresentadas aos participantes. Não se pressupõe equivalência semântica para todas as entradas; cenários adicionais ilustram diferenças de comportamento.”* Isso comunica seu critério sem prometer uma transformação geral que preserve comportamento.

O que os dados permitem concluir

Tempo, tentativas e medidas de olhar permitem comparar desempenho e distribuição da atenção. Sozinhos, não provam que alguém pensou “isso é Python”, “cada item vai para uma posição” ou “a função é automática”. Essas interpretações são mecanismos propostos e precisam de evidência específica para serem afirmadas como resultados.

Esse limite é particularmente visível nos casos 4 e 5, cujos valores escolhidos escondem diferenças de interpretação, e no caso 6, em que ramos diferentes chegam à mesma saída. Uma pessoa pode acertar com uma justificativa tecnicamente incorreta.

Complementos possíveis, sem alterar o protocolo atual

Explicação posterior: perguntar qual regra ou ramo levou à resposta, após a tarefa, para não interferir no tempo e no olhar durante a leitura principal.

Experiência prévia: considerar familiaridade com Python, programação orientada a objetos e RTL como possíveis moderadores das hipóteses.

Reprodutibilidade: registrar códigos exatos, entradas, gabaritos, simulador, versão, opções de execução e revisão dos estímulos; isso é especialmente importante no caso 3.

O rótulo “sem átomo” deve ser entendido como a condição comparativa que torna a regra mais explícita. Ele não significa ausência de toda dificuldade possível. As sugestões acima são para discussão metodológica; nenhuma foi incorporada à aplicação neste trabalho.

Fontes para aprofundamento

As explicações combinam análise dos códigos locais, regras técnicas documentadas e hipóteses propostas na conversa. Os exemplos numéricos e tabelas de percurso foram elaborados para este guia. Os números entre colchetes apontam para as fontes abaixo.

- [1] Sutherland, Stuart; Mills, Don. Standard Gotchas in Verilog and SystemVerilog. SNUG Boston, 2006. Seções 4.1, 4.2 e 5.2-5.3: literais, base, sinal e ampliação. [Abrir fonte](#).
- [2] Rich, Dave. Usage of tick (') in array assignments - packed vs unpacked. Verification Academy, 2018. Distinção entre padrões de atribuição e concatenações; os exemplos da discussão incluem arrays unpacked. [Abrir fonte](#).
- [3] Sutherland, Stuart. Verilog-2001 Quick Reference Guide. Regras herdadas por SystemVerilog para literais, concatenações, operadores, decisões e funções. Não é uma referência para os recursos exclusivos de classes SystemVerilog. [Abrir fonte](#).
- [4] Rich, Dave. Need clarification on static method and non static method. Verification Academy, respostas de 2016 e 2025. Tempo de vida versus método pertencente à classe. [Abrir fonte](#).
- [5] Rich, Dave. Function arguments not initializing variable inside the body. Verification Academy, 2016. Inicialização estática antes do tempo zero e diagnóstico de tempo de vida implícito. [Abrir fonte](#).
- [6] Verification Academy. Signed Arithmetic in Constraints, 2020. Discussão técnica de sinal dos operandos, casts e interpretação de expressões. Usada como apoio às regras, não como origem do exemplo experimental. [Abrir fonte](#).
- [7] Python Software Foundation. The Python Language Reference, Expressions, Comparisons. Encadeamento de comparações e avaliação do termo intermediário. [Abrir fonte](#).
- [8] Sutherland, Stuart; Mills, Don. Can My Synthesis Compiler Do That? DVCon, 2014, seção 10. Curingas e máscara assimétrica; contextualização de casex e das alternativas SystemVerilog. [Abrir fonte](#).
- [9] Is the Simulator Behavior Wrong for My SystemVerilog Code? DVCon, 2017, seção F. O operador ==? pode produzir X; a posição do operando com desconhecido importa. [Abrir fonte](#).
- [10] Para consulta normativa: IEEE Std 1800-2023. A Accellera anuncia o acesso pelo programa IEEE GET. O anúncio fornece o caminho de acesso; não foi tratado como substituto do texto integral da norma neste guia. [Abrir fonte](#).

Proveniência dos exemplos

Trechos conferidos em lambda/app.js e omega/app.js, correspondentes às pastas vscode-pages-Smell e vscode-pages-NoSmell, na revisão de 21/09/2026. O responsável confirmou o resultado do caso 3 no simulador utilizado. Este guia não relata uma execução independente em simulador HDL nem novos resultados com participantes.